

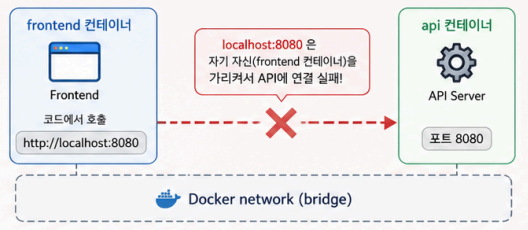
3교시: Readiness Gap과 Transaction Boundary

Week 3 Day 2 Lesson 3 – 컨테이너 네트워크 복습

컨테이너 간 통신은 localhost가 아니라 서비스 이름(DNS)을 사용합니다.

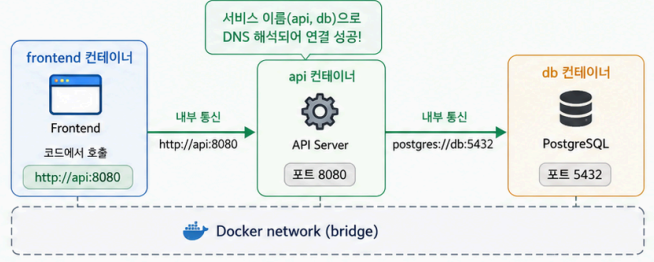
❌ localhost 오해 (잘못된 방식)

컨테이너 안에서 localhost는 자기 자신을 가리킵니다.

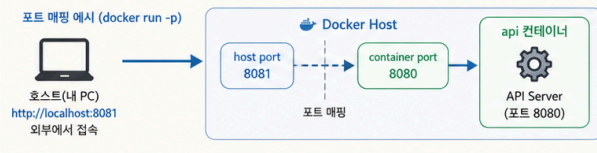


✅ service name DNS (올바른 방식)

Docker 네트워크 내부에서는 서비스 이름으로 서로를 찾습니다.



호스트(외부)와 컨테이너(내부)의 포트 개념



핵심 정리

구분	설명	예시
내부 통신 (컨테이너 ↔ 컨테이너)	Docker network 내부에서 서비스 이름(DNS) 사용	frontend → api:8080 api → db:5432
외부 접속 (호스트 → 컨테이너)	호스트의 포트를 통해 접속 (host port → container port)	localhost:8081 → 컨테이너 8080
localhost의 의미	컨테이너 안에서 localhost는 자기 자신을 의미	컨테이너 내부에서 localhost 사용 금지 (내부 통신 시)

기억할 포인트

- 컨테이너 내부 통신은 localhost가 아닌 서비스 이름 사용
- 외부에서 접속할 때는 host port를 사용
- Docker network가 서비스 이름을 DNS로 해석해줌
- 내부 통신은 container port를 사용

자주 쓰는 내부 DNS 예시

- frontend → api:8080
- api → db:5432
- worker → redis:6379

★ 팁

docker-compose.yml의 service 이름이 곧 DNS 이름입니다.

수업 목표

- /health가 정상이어도 업무 transaction이 안전하다는 뜻을 아님을 설명한다.
- DB, Redis, worker가 모두 살아 있어도 경계 설계가 잘못되면 불일치가 생길 수 있음을 이해한다.
- readiness와 business consistency를 구분한다.
- ghost pending order를 설계 관점으로 분석한다.

D1과 다른 점

D1에서는 health를 이렇게 봤다.

```
api /health 200
order-api /health 200
db healthy
redis running
```

D2에서는 질문이 바뀐다.

모든 dependency가 살아 있을 때만 문제가 없을까?
한 dependency가 요청 중간에 실패하면 어떤 상태가 남을까?
health check가 이런 불일치를 잡아줄까?

Readiness와 일관성은 다르다

개념	질문
readiness	지금 요청을 받을 준비가 되었는가
consistency	요청 처리 후 내부 상태가 모순 없이 남았는가
durability	성공한 상태가 저장되었는가
recoverability	실패 후 복구 또는 재처리가 가능한가

/health는 readiness의 일부를 볼 수 있다. 하지만 transaction boundary가 올바른지는 /health만으로 알 수 없다.

Transaction Boundary 분석

현재 주문 생성 흐름:

```
begin DB work
insert orders
insert order_items
insert audit_logs(order_created)
commit DB
LPUSH redis order-events
return 201
```

위 구조의 위험:

실패 지점	남는 상태	문제
DB insert 전 실패	주문 없음	비교적 단순
DB commit 전 실패	rollback 가능	비교적 단순
DB commit 후 Redis 실패	pending 주문만 남음	ghost pending
Redis 성공 후 response 실패	client는 실패, 내부 처리는 진행	재시도 시 중복 위험
worker 처리 후 response 없음	client는 모름, 업무는 완료	상태 조회 필요

설계 대안

방식	장점	단점
현재 방식	이해하기 쉽고 구현이 단순	DB/queue 불일치 가능
DB transaction 안에 outbox 기록	DB 상태와 event 발행 의도를 같이 저장	publisher 추가 필요
Redis publish 먼저	queue event는 생김	DB 주문 row가 없을 수 있음
API에서 동기 처리	완료 기준 단순	응답 지연과 실패 전파 증가
idempotency key	재시도 안전성 증가	저장소와 정책 필요

운영 판단 질문

Ghost pending order가 발견되면 바로 삭제하면 안 된다.

질문	이유
사용자가 실패 응답을 봤는가	재시도 가능성이 있다
결제가 붙는 주문인가	임의 삭제 위험이 크다
queue event를 재발행할 수 있는가	복구 방식 결정
idempotency key가 있는가	중복 처리 방지 가능 여부
audit에 어떤 단계까지 남았는가	업무 단계 판단

실습 확장

2교시 스크립트 결과를 가지고 다음 표를 채운다.

항목	결과
client response	
DB order exists	
queue event exists	
audit order_created	
audit order_processed	
자동 복구 가능 여부	
수동 조치 후보	

Kubernetes 연결

Kubernetes readinessProbe는 "준비되지 않은 pod로 traffic을 보내지 않는 데" 도움을 준다. 하지만 DB commit 후 Redis publish 실패 같은 transaction boundary 문제는 Kubernetes가 자동으로 해결하지 않는다.

문제	Kubernetes가 도와주는가
죽은 container 재시작	도움 됨
ready 아닌 pod 제외	도움 됨
DB/queue atomic transaction	직접 해결하지 않음
중복 주문 방지	application 설계 필요
ghost pending 복구	job/runbook/application 설계 필요

핵심 포인트

운영 가능한 MSA는 health check만으로 만들어지지 않는다.

readiness는 traffic을 받을 준비
 consistency는 업무 상태가 모순 없이 남는 성질
 recoverability는 깨진 상태를 다시 수습할 수 있는 성질

Evidence Note

```
# W3D2S3 Readiness Gap
- health before incident:
- failure boundary:
- committed state:
- missing event:
- consistency risk:
- Kubernetes helps:
- application must solve:
```